



Data Science and Management

Corso di Laurea Magistrale in Ingegneria Gestionale

Marco Mamei, Natalia Selini Hadjidimitriou
{marco.mamei, selini}@unimore.it

- Decision Trees: tree-based models for classification and regression, explaining structure, decision rules, and the C4.5 algorithm.
- Attribute Selection: metrics like Gini Index and Information Gain (with Gain Ratio), entropy concepts, and pruning techniques to avoid overfitting.
- Random Forests, ensemble methods (bagging, boosting, stacking), and Scikit-Learn examples with the Wine dataset.

What Are Decision Trees?



Decision Trees are intuitive models for classification and regression.

Represent decisions as a tree structure.

Internal nodes: attributes; branches: values; leaves: classes.

Decision Trees



Decision trees are one of the most simple yet widely used machine learning methodologies

- Given attribute/value **representations**
- Given a **target property to classify**
- Given a training set, inductively learn a **tree**
- Each **internal** node of the tree is an **attribute**
- Each branch is a **value** (or a range) for that attribute
- Each leaf is one of the **classes**

Decision Trees



Example – Golf Decision Tree

"Shall we play golf?"

Attributes: Outlook, Temperature, Humidity, Wind.

Target: Play or Not Play.

Decision Rules

Each path from root to leaf = a classification rule.

Example: If Outlook = Sunny AND Humidity = High → Don't Play.

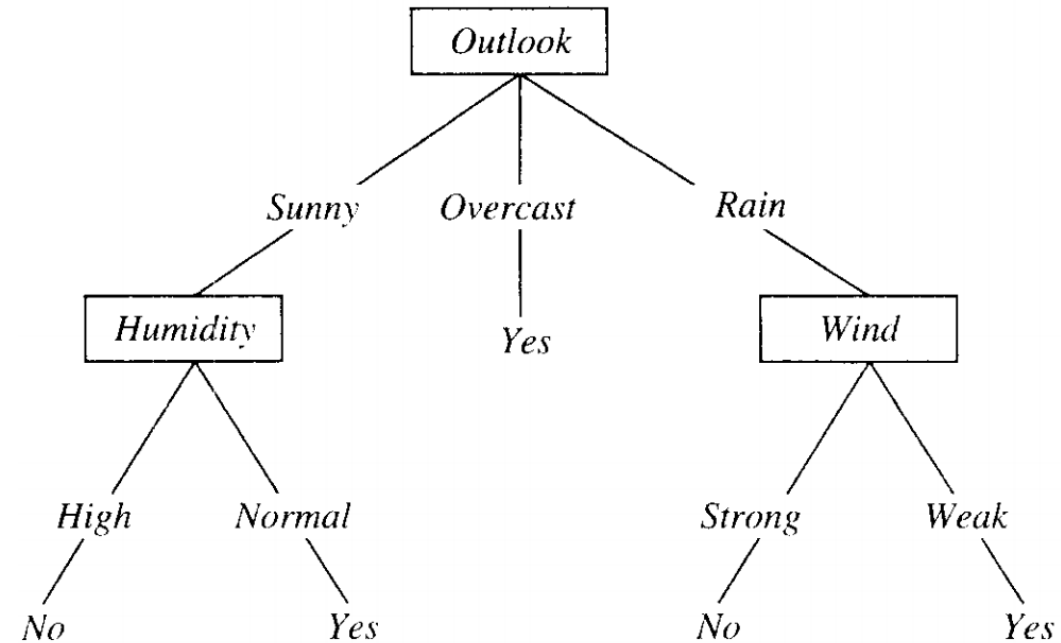
Decision Trees

Shall we play golf?

ID	ATTRIBUTES				TARGET
No	Outlook	Temp (° F)	Humid (%)	Windy	Class
D1	sunny	75	70	T	P
D2	sunny	80	90	T	N
D3	sunny	85	85	F	N
D4	sunny	72	95	F	N
D5	sunny	69	70	F	P
D6	overcast	72	90	T	P
D7	overcast	83	78	F	P
D8	overcast	64	65	T	P
D9	overcast	81	75	F	P
D10	rain	71	80	T	N
D11	rain	65	70	T	N
D12	rain	75	80	F	P
D13	rain	68	80	F	P
D14	rain	70	96	F	P

Decision Trees

Shall we play golf?



[Image by Tom Mitchell]

A path from root to leaf is a **classification rule**

Decision Trees



Many algorithms exist that **learn** decision trees
One of the most famous is C4.5 by [R. Quinlan, 93]

C4.5 Algorithm Steps

- Choose best attribute A.
- Partition dataset by A's values.
- Recursively apply C4.5 to subsets.
- Stop when subset is pure or nearly pure.

Decision Trees



Attribute Selection Criteria

- Perfect attribute: splits data into pure subsets.
- Useless attribute: no change in class distribution.
- Use information theory to measure effectiveness.

Decision Trees



Possible metrics employed for attribute selection:

- Gini Index (to minimize impurity)
- Information Gain Ratio (to maximize entropy reduction)

Decision Trees: Gini Index

Gini Index Explained

- Measures impurity of a node.
- Lower Gini = purer node.
- Used to select best attribute for splitting.

Gini Index Properties

- Value ranges from 0 (pure) to 1 (impure).
- Easy to interpret.
- Can be misleading with attributes having many values.

Why Gini Works

- Pure nodes have $Gini = 0$.
- Impure nodes have higher Gini.
- Helps identify meaningful splits.

Decision Trees: Gini Index



Gini index or Impurity measure

- measures the degree or probability of a particular variable being wrongly classified when it is randomly chosen

Impurity:

- If all the elements belong to a single class, then it can be called pure

The degree of Gini Index varies between 0 and 1:

- '0' denotes that all elements belong to a certain class or there exists only one class (pure)
- '1' denotes that the elements are randomly distributed across various classes (impure)

Decision Trees: Gini Index

Gini index (to be **minimized**)

- Perform a split: Divide node **N** into **B** child nodes based on an attribute
- **Compute Gini for the split** using:

$$G(S) = \sum_{i=1}^B \rho_i \left(1 - \sum_{k=1}^K p_{ik}^2 \right)$$

Where:

- p_i = proportion of examples from node **N** that go into child **i**
- p_{ij} = proportion of examples of class **j** in child **i**
- C = number of classes

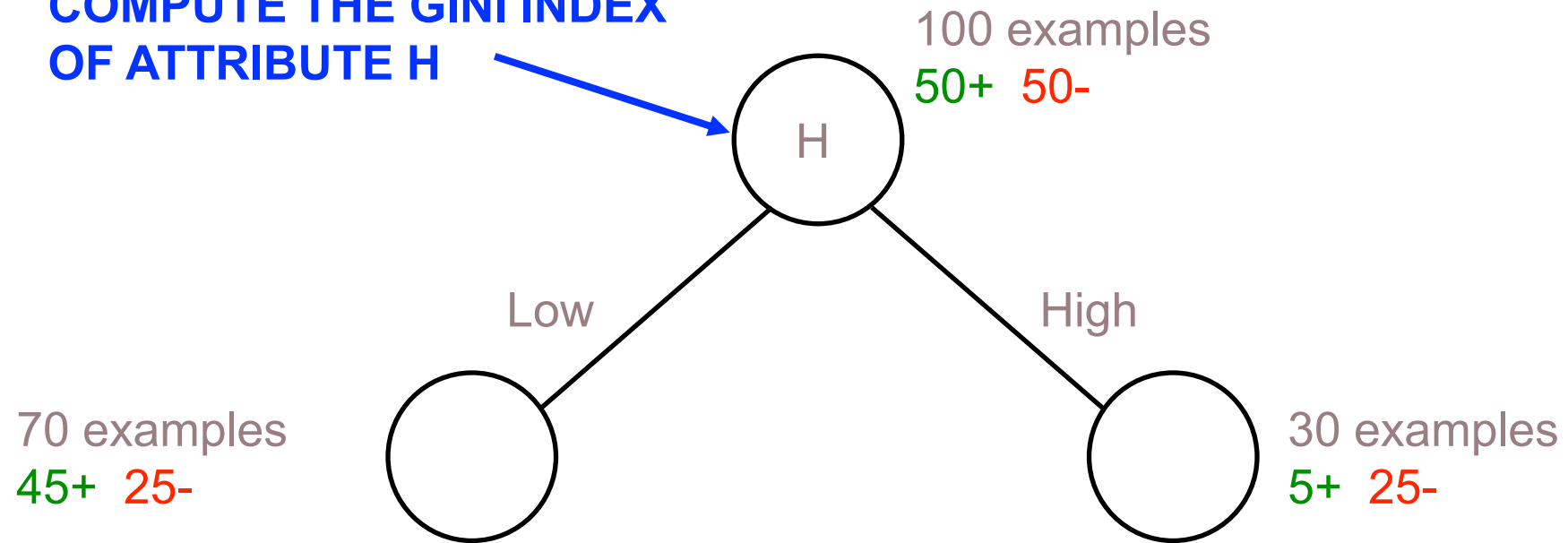
Interpretation:

- If all examples in a child node belong to one class → Gini = 0 (pure).
- If classes are evenly distributed → Gini is higher (impure).

Goal: Choose the attribute that produces the **smallest Gini index**, because it creates the purest split.

Decision Trees: Gini Index

COMPUTE THE GINI INDEX
OF ATTRIBUTE H



$$G(H) = 70/100 * (1 - (45/70)^2 - (25/70)^2) + 30/100 * (1 - (5/30)^2 - (25/30)^2)$$

Decision Trees: Gini Index



Advantage of Gini index

- Node purity is **easily interpretable**

Drawback of Gini index

- Attributes with **many values** can induce a **not meaningful** split with a very high purity

Example: fiscal code of a person

- Better to use it with attributes with **few values**

Decision Trees: Information Gain



Measures reduction in entropy after a split.
Higher gain = better attribute.

Entropy Explained

- Entropy = measure of uncertainty.
- Maximized when outcomes are equally likely.

Information Gain Ratio

- Normalizes Information Gain by attribute entropy.
- Used in C4.5 to avoid bias toward many-valued attributes.

Gain Ratio – Pros and Cons

- Corrects bias of Information Gain.
- May favor low-entropy attributes unfairly.

Decision Trees: Information Gain

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \cdot H(S_v)$$

Where:

- $H(S)$: Entropy of the original dataset S
- S_v : Subset of S after splitting by attribute A
- $|S_v| / |S|$: Proportion of examples in subset S_v
- $H(S_v)$: Entropy of subset S_v

Interpretation:

- Information Gain measures reduction in entropy after a split.
- Higher IG means better attribute for splitting.
- Goal: Maximize IG to reduce uncertainty in data.

Decision Trees

A note on entropy

- It measures **disorder** in data, that is the average **information** in a variable's outcome
- Highest value when **uncertainty** is highest

Example: if tossing a coin the entropy of the outcome is maximum when heads and tails are equally probable

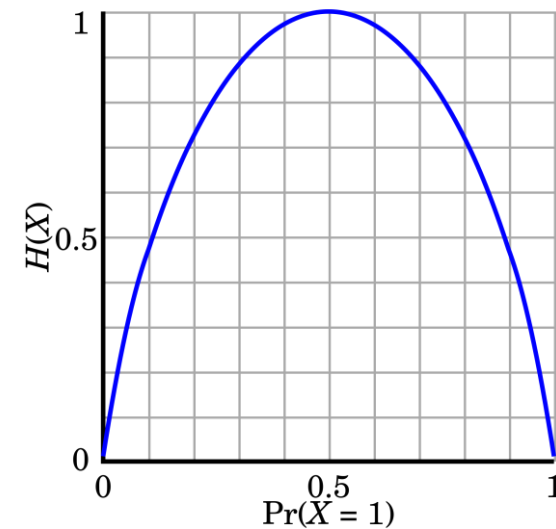
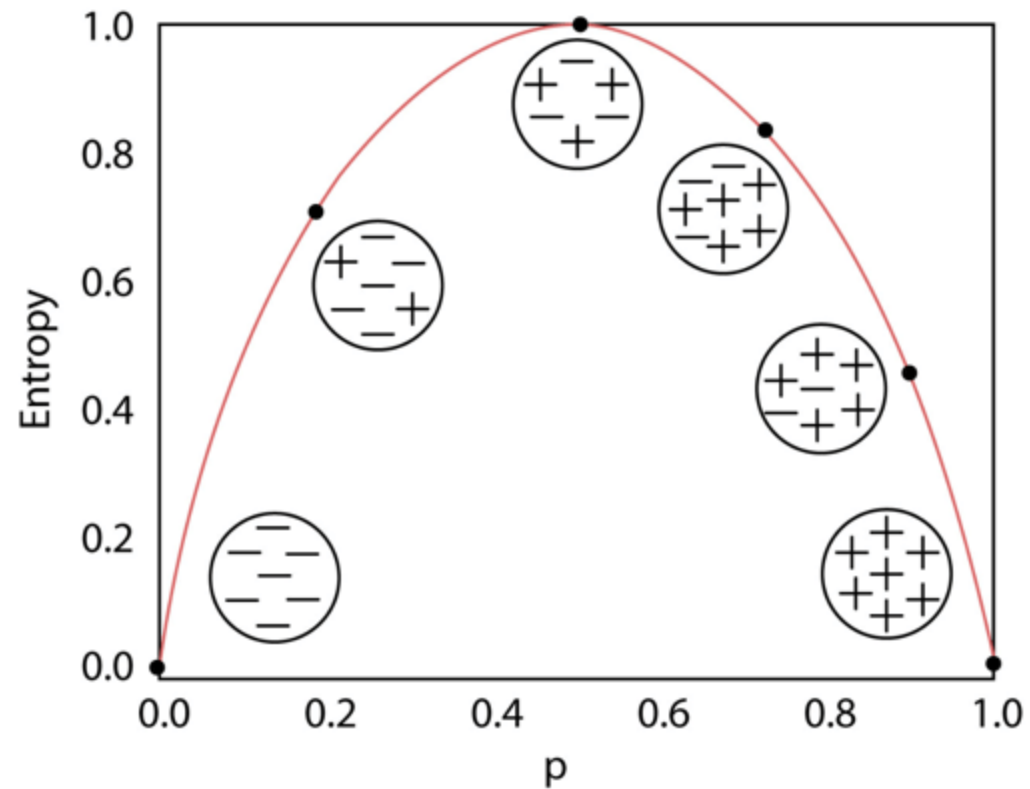


Figure from Wikipedia

Decision Trees

Variation of entropy against data points:



By Chainika Thakar and Shagufta Tahsildar

Decision Trees

Information Gain Ratio **USED IN C4.5**

- The solution is to normalize the Information Gain by the **entropy of the attribute**

$$IGR(S) = \frac{IG(S)}{H_A(N)}$$

where $H_A(S) = - \sum_{j=1}^{|A|} p_j \log_2(p_j)$

being p_j the % of examples with value j for A

Decision Trees



Information Gain Ratio

- **Advantage:** it solves the problem with Information Gain and Gini regarding attributes with many values
- **Disadvantage:** it may give advantage to attributes having a low information gain (just because they have a low entropy)

Decision Trees



Decision trees can easily **overfit** data

- **Pre-pruning:** Stop growing tree when:
 - Node is pure or nearly pure.
 - Few examples remain.
 - No gain in purity.
- **Post-pruning:** Build full tree, then prune:
 - Remove subtrees that don't improve validation accuracy.
 - Prefer simpler trees (Occam's Razor).

Decision Trees



How to handle continuous variables

- Try all possible split values.
- Sort values to improve efficiency.
- Apply same metrics (Gini, Gain).

How to handle missing values...

- Ignore missing values in metric computation.
- Assign examples to majority branch or weight all branches.

Decision Trees



Pros

- Simplicity
- Interpretability — **glass box**
- Can handle **missing values**
- Can handle **continuous and categorical** variables

Cons

- Highly prone to **overfitting**
- High variance with **small changes** in training sets

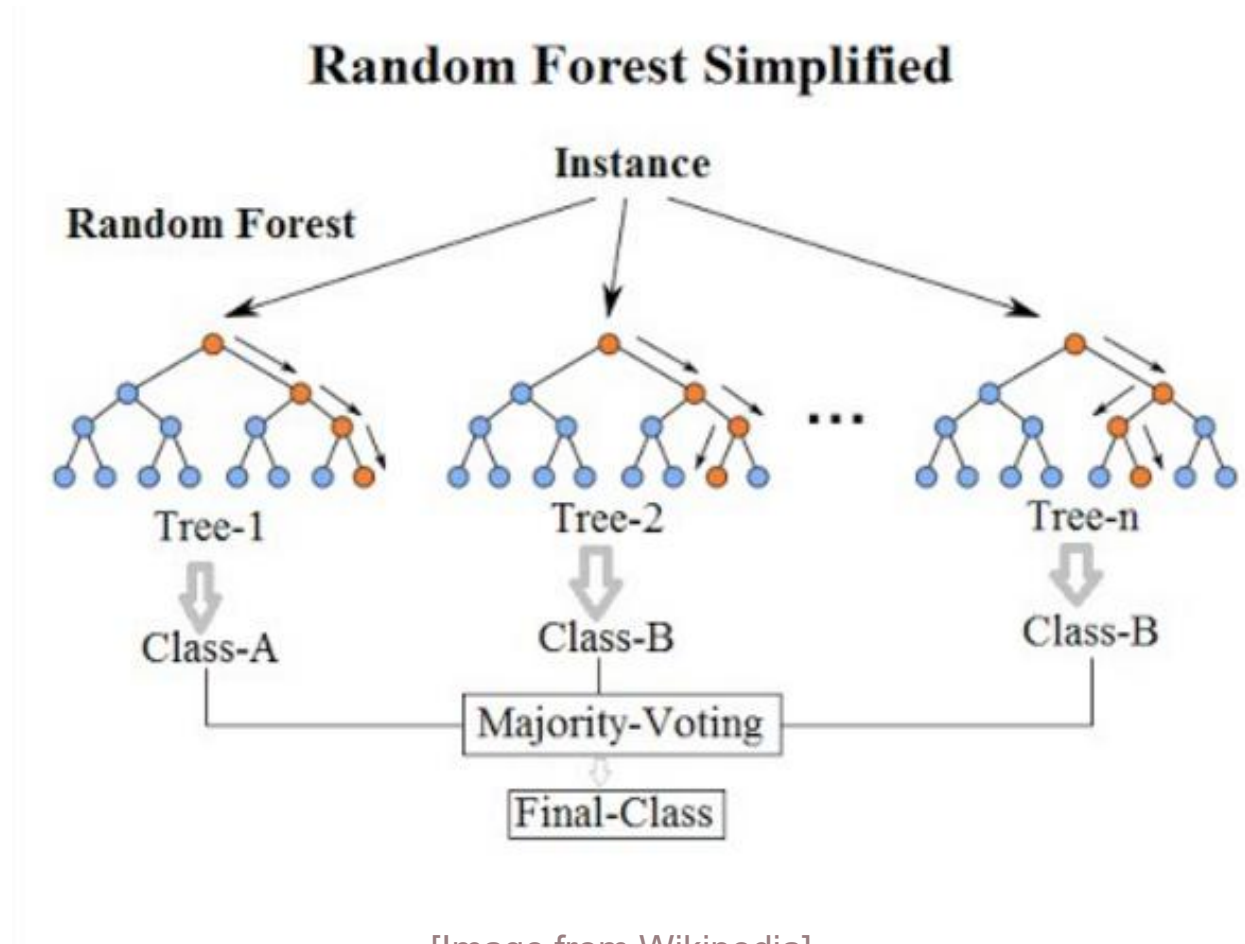
Decision Trees: Random Forests



A very effective extension: **Random Forests**

- Many **different** decision trees
- Each learned on a **subset** of the attributes
- **Average** prediction over all trees
- Named an **ensemble** method
- Typically **higher performance** than decision trees

Random Forests



Ensemble Methods



Combining weak classifiers into stronger ones: often this brings an improvement in terms of performance

- Bagging
- Boosting
- Stacking

Ensemble Methods



Bagging

- Homogeneous weak learners
- Learning in parallel
- Somehow aggregate the results

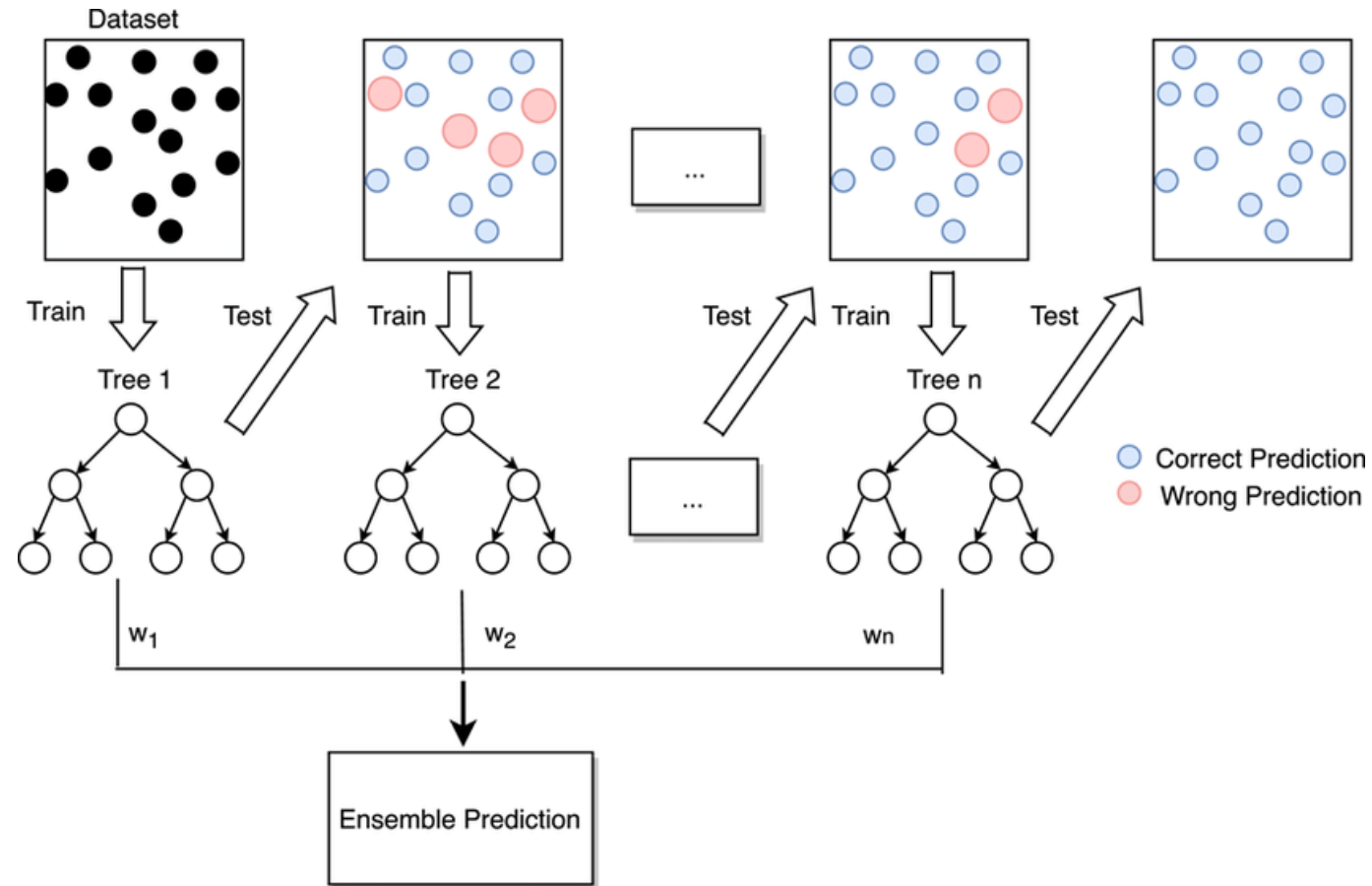
Ensemble Methods



Boosting

- Homogeneous weak learners
- Learning sequentially
- Combination with some deterministic strategy

Gradient Boosting



[Image from Zhang et al.]

Ensemble Methods



Stacking

- Heterogeneous weak learners
- Learning in parallel
- Feed predictions of several models into a higher level model (meta-model)

Example with Scikit-Learn



Example — Wine dataset

Data collected from chemical analysis of wines grown in the same region in Italy but derived from three different cultivars.

The analysis determined the quantities of 13 constituents found in each of the three types of wines.

Example with Scikit-Learn

Example — Wine dataset

List of attributes:

1. Alcohol
2. Malic acid
3. Ash
4. Alcalinity of ash
5. Magnesium
6. Total phenols
7. Flavanoids
8. Nonflavanoid phenols
9. Proanthocyanins
10. Color intensity
11. Hue
12. OD280/OD315 of diluted wines
13. Proline

Example with Scikit-Learn

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from sklearn.tree import DecisionTreeClassifier

X, y = datasets.load_wine(return_X_y=True)
X_trainval, X_test, y_trainval, y_test = train_test_split(X, y,
test_size=0.2)
X_train, X_val, y_train, y_val = train_test_split(X_trainval,
y_trainval, test_size=0.3)

dt = DecisionTreeClassifier()
dt.fit(X_trainval, y_trainval)
y_pred = dt.predict(X_test)
print(classification_report(y_test, y_pred))
```

Example with Scikit-Learn

Print textual version

```
from sklearn import tree
print(tree.export_text(dt))
```

Print with “tree” library

```
fig = plt.figure()
tree.plot_tree(dt, filled=True)
fig.savefig("decision_tree.pdf")
```

Print via Graphviz

```
dot_data = tree.export_graphviz(dt, filled=True)
pydot_graph = pydotplus.graph_from_dot_data(dot_data)
pydot_graph.write_pdf('decision_tree.pdf')
```

Example with Scikit-Learn

Random Forest

```
from sklearn.ensemble import RandomForestClassifier
dt = RandomForestClassifier()
dt.fit(X_trainval, y_trainval)
y_pred = dt.predict(X_test)
print(classification_report(y_test, y_pred))
```

Gradient Boosting

```
from sklearn.ensemble import GradientBoostingClassifier
dt = GradientBoostingClassifier()
dt.fit(X_trainval, y_trainval)
y_pred = dt.predict(X_test)
print(classification_report(y_test, y_pred))
```